

Function-Oriented Software Design

Introduction

- Function-oriented design techniques are popular:
 - Used in software development organizations, even today.
- Function-oriented design techniques:
 - Start with functional requirements specified in SRS document.
- During the design process:
 - High-level functions are successively decomposed into more detailed functions.
 - Successive decomposition of high-level functions into more detailed functions known as **Top-down decomposition.**
 - Finally detailed functions are mapped to a module structure.

SA / SD

(Structured Analysis/Structured Design)

- SA/SD technique used to perform high-level design.
- SA/SD methodology:
 - has essential features of several important function-oriented design methodologies
 - If you need to use any specific design methodology later on, you can do so easily with small additional effort.
- SA/SD technique draws heavily from the following methodologies:
 - Constantine and Yourdon's methodology
 - Hatley and Pirbhai's methodology
 - Gane and Sarson's methodology
 - DeMarco and Yourdon's methodology

Overview of SA/SD Methodology

- SA/SD consists of two distinct activities:
 - Structured Analysis (SA)
 - Structured Design (SD)
- During **Structured Analysis**:
 - Functional decomposition takes place as follows.
 - Each function is analyzed - Hierarchically decomposed into more detailed functions.
 - Simultaneous decomposition of high-level data into more detailed data.
 - Transforms a textual problem description into a graphic model.
 - Done using data flow diagrams (DFDs) (or bubble charts).
 - DFDs graphically represent the results of structured analysis.
- During **Structured Design**:
 - System module structure is formalized.
 - All the functions represented in the DFD mapped to a module structure (aka module structure chart).

SA vs. SD

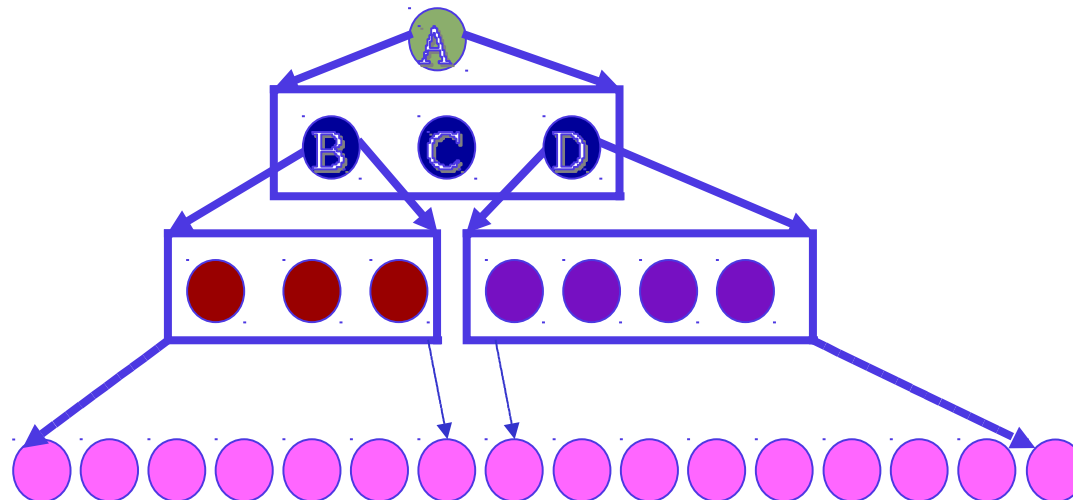
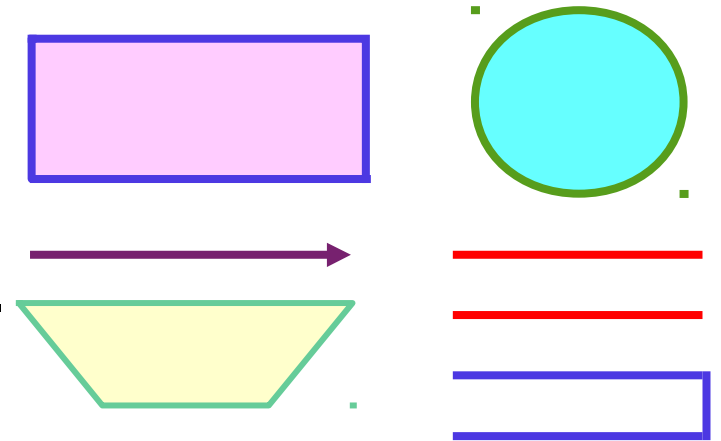
- Purpose of structured analysis:
 - Capture the detailed structure of the system as the user views it.
- Purpose of structured design:
 - Arrive at a form that is suitable for implementation in some programming language.
- The results of **structured analysis** can be easily understood even by ordinary customers:
 - Does not require technical knowledge.
 - Directly represents customer's perception of the problem.
 - Uses customer terminology for naming different functions & data.
 - The results of **structured analysis** can be reviewed by customers:
 - To check whether it captures all their requirements.

Data Flow Diagrams

- DFD is an elegant modelling technique:
 - Useful not only to represent the results of structured analysis.
 - Applicable to other areas also:
 - e.g. for showing the flow of documents or items in an organization,
- DFD technique is very popular technique.
 - It is powerful and yet simple to understand and use.
- DFD is a hierarchical graphical model.
 - Shows the different functions (or processes) of the system, and
 - Data interchange among the processes.
- Useful to consider each function as a processing station.
 - Each function consumes some input data.
 - Produces some output data.

Data Flow Diagrams (DFDs)

- A DFD model:
 - Uses limited types of symbols.
 - Simple set of rules
 - Easy to understand - It is a hierarchical model.
 - In a hierarchical model,
 - We start with a very simple and abstract model of a system,
 - Details are slowly introduced through the hierarchies.



External Entity Symbol

Librarian

- Represented by a rectangle
- External entities are real physical entities - input data to the system or consume data produced by the system.
 - Sometimes external entities are called **terminator, source, or sink**.

Function Symbol

Search
book

- A function such as “Search book” is represented using a circle:
 - This symbol is called a **process** or **bubble** or **transform**.
 - Bubbles are annotated with corresponding function names.
 - Functions represent some activity - **Function names should be verbs**.

Data-flow Symbol

→
Book-name

- A directed arc or line.
 - Represents data flow in the direction of the arrow.
 - Data flow symbols are annotated with names of data they carry.

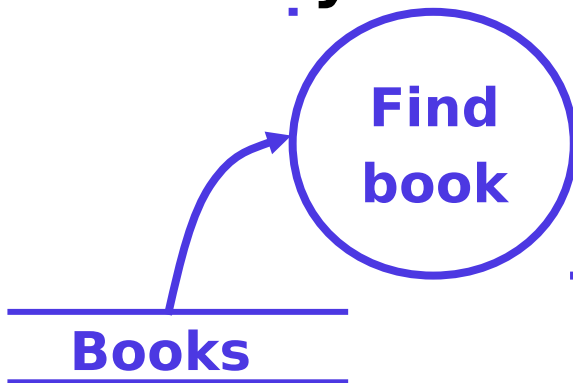
Data Store Symbol

book-details



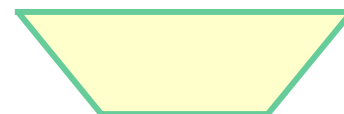
- Represents a logical file:
 - A logical file can be a data structure, or a physical file on disk.
 - Each data store is connected to a process by means of a data flow symbol.

Data Flow and Store Symbol



- Direction of data flow arrow shows whether **data is being read from or written into** it.
- An arrow into or out of a **data store**:
 - Implicitly represents the entire data of that data store
 - **Arrows connecting to a data store need not be annotated with any data name.**

- Output produced by the system



**Output
Symbol**

